

Thread Synchronization in C# with Examples

In this article, I am going to discuss **Thread Synchronization in C#** with Examples. Please read our previous article where we discussed the significance of the [IsAlive Property and Join Method of Thread Class in C#](#) with Examples. At the end of this article, you will understand what is thread synchronization and how you can implement thread synchronization to protect shared resources in multithread applications with real-time examples.

What is Thread Synchronization in C#?

Data inconsistency occurs when more than one threads access a shared resource such as in-memory data (instance or class variables) and external objects such as files at the same time. Let us understand this with an example. Consider that we have two threads Thread1 and Thread2, and both the threads access a shared resource let's say Resource1 simultaneously. If Thread1 is trying to read data from the shared resource Resource1 when Thread2 is attempting to write data onto the shared resource Resource1, then there would be data inconsistency. Hence, in situations like this thread synchronization comes into the picture.

Synchronization in C# language is a process that allows access to shared resources smoothly. Synchronization in C# ensures that only one thread is accessing the shared resource at any given point in time, preventing other threads from doing the same at the same time.

Thread Synchronization in C# is a mechanism that is used to restrict multiple threads from accessing a shared resource at the same time. In simple words, we can also say that thread synchronization can help us to prevent multiple threads from gaining access to a shared resource simultaneously. As a result, we can have one and only one thread entering a critical section to access the shared resource at any given point in time.

How Thread Synchronization is Achieved in C#?

Synchronization in C# can be achieved in multiple ways. One of the ways to achieve Synchronization in C# is by using the feature of lock, which locks the access to a block of code within the locked object. When a thread locks an object, no other thread can access the block of code within the locked object. Only when a thread releases the lock, then it is available for other threads to access it.

In C# Language, every object has a built-in lock. By using the feature of Synchronization, we can lock an object. Locking an object can be done by using the lock keyword, and the following is the syntax to use the lock.

```
lock(object)
{
    //Statement1
    //Statement2
    //And more statements to be synchronized
}
```

So, when a thread acquires a lock over an object, then that particular thread can only access the block of statements within the locked object. Now, all the other threads wishing to access the same block of statements within the same locked object will have to wait until, the thread that has got the lock on the object, releases the lock, by exiting the block of statements.

Example without Thread Synchronization in C#:

Before we show you the example of how to use the synchronization between threads by locking an object and its practical use, let us first see what actually happens without using synchronization on executing multiple threads, which are trying to access the same resource.

In the below example, we are creating three different threads that are going to access the same resource i.e. in this case the shared resource is `SomeMethod`. As you can see, we have delayed the `SomeMethod` execution by 1 second, and all three threads trying to execute the same method and they will execute it. The first thread which executes the method does not get its sole access, this thread executes the method for a while and after some time, other threads will come and execute the same method. So, here, we will not get the output as expected.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            Thread thread2 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            Thread thread3 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            thread1.Start();
            thread2.Start();
            thread3.Start();
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            Console.Write("[Welcome To The ");
            Thread.Sleep(1000);
            Console.WriteLine("World of Dotnet!"]);
        }
    }
}
```

When I run the above code, I am getting the following output. The output might be varied in your machine.

```
[Welcome To The [Welcome To The [Welcome To The World of Dotnet!]
World of Dotnet!]
World of Dotnet!]
```

As you can see in the above output, here we are not getting the output as expected. So, the point that you need to keep in mind is that if the shared resource is not protected in a multithreaded environment from concurrent access, then the output or the behavior of the application becomes inconsistent.

Example using Thread Synchronization in C#

In the below example, we are creating three threads that are going to access the SomeMethod, but this time access to SomeMethod will be synchronized because we are going to use the lock, to lock the object within which the method is going to be accessed by multiple threads. The first thread to enter the method gets its sole access until it exits the method, thereby avoiding the collision between multiple threads trying to access a method.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            Thread thread2 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            Thread thread3 = new Thread(SomeMethod)
            {
                Name = "Thread 2"
            };
            thread1.Start();
            thread2.Start();
            thread3.Start();
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            // Locking the Shared Resource for Thread Synchronization
            lock (lockObject)
            {
                Console.Write("[Welcome To The ");
                Thread.Sleep(1000);
                Console.WriteLine("World of Dotnet!]");
            }
        }
    }
}
```

```

    }
}
}

```

Output:

```

[Welcome To The World of Dotnet!]
[Welcome To The World of Dotnet!]
[Welcome To The World of Dotnet!]

```

The first thread enters the SomeMethod locked the method (code written within the lock (object)), and gets its sole access and once this thread has finished its execution of the method, then only another thread will come and acquire a lock on the critical section i.e. the code written within the lock object. In this way, the lock object will make sure that at any given point in time, only one thread can access the shared resource which will also ensure data consistency.

Realtime Example to Understand Thread Synchronization in C#:

Let us see one real-time example to understand thread synchronization and why it is important to protect shared resources in our application. In the below example, I am going to show you how to protect a shared variable from concurrent access in a multithread environment.

In the below example, we are implementing the ticket booking functionality for a movie. Let us assume the number of available tickets is 3 and three different threads tried to book tickets. Thread1 tries to book 1 ticket, thread2 tries to book 2 tickets and thread3 tries to book 3 tickets. Let us first see the problem without thread synchronization. In the below code, we are not implementing thread synchronization and hence sometime you will see that all three threads are able to book tickets. See, the available tickets are 3 and we are able to book 6 tickets and this is the issue without thread synchronization.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            BookMyShow bookMyShow = new BookMyShow();
            Thread t1 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread1"
            };
            Thread t2 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread2"
            };
            Thread t3 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread3"
            };

```

```

        };

        t1.Start();
        t2.Start();
        t3.Start();
        Console.ReadKey();
    }
}

public class BookMyShow
{
    int AvailableTickets = 3;
    static int i = 1, j = 2, k = 3;
    public void BookTicket(string name, int wantedtickets)
    {
        if (wantedtickets <= AvailableTickets)
        {
            Console.WriteLine(wantedtickets + " booked to " + name);
            AvailableTickets = AvailableTickets - wantedtickets;
        }
        else
        {
            Console.WriteLine("No tickets Available to book");
        }
    }
    public void TicketBookig()
    {
        string name = Thread.CurrentThread.Name;
        if (name.Equals("Thread1"))
        {
            BookTicket(name, i);
        }
        else if (name.Equals("Thread2"))
        {
            BookTicket(name, j);
        }
        else
        {
            BookTicket(name, k);
        }
    }
}
}

```

Output:

```

2 booked to Thread2
1 booked to Thread1
3 booked to Thread3

```

As you can see in the above output, all the threads are able to book the tickets. This is possible because all the threads are able to access the critical section code of the program simultaneously. Now, let us proceed and see how we can restrict this i.e. how we can allow only one thread to execute the critical section code.

Real-Time Example using Thread Synchronization in C#

In the below example, we have used the thread synchronization mechanism to lock the critical section code by using the lock object. Now, the lock object will make sure that only one thread can execute the critical section code, and once the thread completes the execution of the critical section code, then another thread can enter and execute the critical section code. With this, now you will get the output as expected and you will see that you cannot book more than 3 tickets.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            BookMyShow bookMyShow = new BookMyShow();
            Thread t1 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread1"
            };
            Thread t2 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread2"
            };
            Thread t3 = new Thread(bookMyShow.TicketBookig)
            {
                Name = "Thread3"
            };

            t1.Start();
            t2.Start();
            t3.Start();
            Console.ReadKey();
        }
    }
    public class BookMyShow
    {
        private object lockObject = new object();
        int AvailableTickets = 3;
        static int i = 1, j = 2, k = 3;
        public void BookTicket(string name, int wantedtickets)
        {
            lock(lockObject)
            {
                if (wantedtickets <= AvailableTickets)
```

```

        {
            Console.WriteLine(wantedtickets + " booked to " + name);
            AvailableTickets = AvailableTickets - wantedtickets;
        }
        else
        {
            Console.WriteLine("No tickets Available to book");
        }
    }
}
public void TicketBookig()
{
    string name = Thread.CurrentThread.Name;
    if (name.Equals("Thread1"))
    {
        BookTicket(name, i);
    }
    else if (name.Equals("Thread2"))
    {
        BookTicket(name, j);
    }
    else
    {
        BookTicket(name, k);
    }
}
}
}
}

```

Output:

```

1 booked to Thread1
2 booked to Thread2
No tickets Available to book

```

So, by using the feature of synchronization (lock), we can avoid a conflict between threads trying to access the same resource simultaneously. We can achieve thread synchronization in C# by using the followings. From our next article onwards, we are going to discuss the following in detail.

1. **Lock**
2. **Monitor**
3. **Mutex**
4. **Semaphore**
5. **SemaphoreSlim**

Note: Thread Synchronization in C# is a mechanism that ensures that two or more concurrent processes or threads do not execute some particular section of the program, especially the critical section. In this technique, one thread executes the critical section of a program and the other thread waits until the thread finished its execution. If a proper synchronization mechanism will be not applied then race conditions will happen.

Why do we need Thread Synchronization in Multithreading?

We need Thread Synchronization in Multithreading because of the following:

1. **Atomicity:** Thread Synchronization supports atomicity, which ensures that multiple threads in the application are not allowed to access a shared resource concurrently to prevent data inconsistency. The code section of our program which causes data inconsistency is known as the critical section. The critical section of our program is executed atomically by one and only one thread which ensures Atomicity. The Ticket booking example is an example of Atomicity.
2. **Ordering:** We generally want two or more threads to perform a task in a particular order or we want to restrict access to shared resources to a particular number of threads only. Usually, we don't have much control over all this, which is one reason for race conditions. Thread synchronization provides support for ordering so that you can have control over your threads to perform the tasks as per your requirement. The first example that we discussed in this article is the example of ordering.

What is Exclusive Lock and Non-Exclusive Lock in C#?

When a process or a thread wants to access an object, it requests a lock on that object. There are two types of locks that determine access to shared resources – Exclusive Lock and Non-Exclusive lock.

1. **Exclusive Lock:** An exclusive lock makes sure that only one thread can gain access or enter a critical section at any given point in time. In C#, we can implement Exclusive Lock using the lock keyword, Monitor class, Mutex Class, and SpinLock class.
2. **Non-Exclusive Lock:** Non-Exclusive locks provide read-only access to a shared resource and limit concurrency, i.e., limit the number of concurrent accesses to a shared resource. In C#, we can implement Non-Exclusive Lock using the Semaphore, SemaphoreSlim, and ReaderWriterLockSlim classes.

In the next article, I am going to discuss [Thread Synchronization using Lock in C#](#) with Examples. Here, in this article, I try to explain **Thread Synchronization in C#** with Examples. I hope you enjoy this Thread Synchronization in C# with Examples article.